

Lecture 3: Sorting Algorithms

CS301 — Algorithms & Data Structures

Department of Computer Science

1. Comparison-Based Sorting

Most sorting algorithms compare pairs of elements. The fundamental lower bound for comparison-based sorting is $\Omega(n \log n)$. Any comparison-based sort must do at least $n \log n$ comparisons in the worst case.

Proof sketch: there are $n!$ possible orderings of n elements. A comparison tree with k leaves can distinguish at most 2^k orderings. So $2^k \geq n! \rightarrow k \geq \log_2(n!) = \Theta(n \log n)$ by Stirling's approximation.

2. Insertion Sort

Insertion Sort: build sorted portion one element at a time. for $i = 1$ to $n-1$: $\text{key} = A[i]$ $j = i - 1$ while $j \geq 0$ and $A[j] > \text{key}$: $A[j+1] = A[j]$ $j--$ $A[j+1] = \text{key}$

Time: $O(n^2)$ worst/average, $O(n)$ best (already sorted) Space: $O(1)$ in-place Stable: Yes

Best for: small arrays, nearly sorted data (online sorting).

3. Merge Sort

Merge Sort: divide and conquer. $\text{mergeSort}(A, l, r)$: if $l \geq r$: return $\text{mid} = (l + r) // 2$ $\text{mergeSort}(A, l, \text{mid})$ $\text{mergeSort}(A, \text{mid}+1, r)$ $\text{merge}(A, l, \text{mid}, r)$

$\text{merge}()$: combine two sorted halves into one sorted array — $O(n)$ time, $O(n)$ space.

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$ by Master Theorem (Case 2).

Time: $O(n \log n)$ all cases Space: $O(n)$ auxiliary Stable: Yes

Best for: linked lists (no extra space needed), external sorting (disk), when stability required.

4. Quicksort

Quicksort: divide and conquer with partitioning. $\text{quickSort}(A, l, r)$: if $l \geq r$: return $p = \text{partition}(A, l, r)$ // pivot ends up at index p $\text{quickSort}(A, l, p-1)$ $\text{quickSort}(A, p+1, r)$

Lomuto partition scheme: choose $A[r]$ as pivot, place all smaller elements before it. Hoare partition scheme: two pointers from both ends, more efficient in practice.

Time: $O(n \log n)$ average, $O(n^2)$ worst (sorted input with bad pivot) Space: $O(\log n)$ average (recursion stack), $O(n)$ worst Stable: No (Lomuto)

Pivot strategies: - First/last element: $O(n^2)$ for sorted input - Random pivot: expected $O(n \log n)$ regardless of input - Median-of-three: pick median of first, middle, last — good in practice

Quicksort is typically faster than Merge Sort due to cache efficiency and smaller constants, even though worst-case is $O(n^2)$. Most library sorts (Python's Timsort, C++'s introsort) are hybrids.

5. Counting Sort and Radix Sort

Non-comparison sorts can beat the $O(n \log n)$ lower bound when keys have special structure.

Counting Sort: when keys are in range $[0, k]$. 1. Count frequency of each key 2. Compute cumulative counts (prefix sum) 3. Place each element in correct position

Time: $O(n + k)$ Space: $O(n + k)$ Stable: Yes

Radix Sort: sort by each digit/character position. for $d = \text{least_significant}$ to most_significant : stable_sort by digit d (use Counting Sort)

Time: $O(d \times (n + k))$ where $d = \text{number of digits}$, $k = \text{digit range}$ Stable: Yes

Radix Sort beats $O(n \log n)$ when d is small (e.g., 32-bit integers: $d=32$, $k=2 \rightarrow O(n)$).

6. Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
-----	-----	-----	-----	-----	-----
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes
Radix Sort	$O(dn)$	$O(dn)$	$O(dn)$	$O(n+k)$	Yes

Timsort (Python's sort): hybrid of Merge Sort + Insertion Sort. $O(n \log n)$ worst, $O(n)$ best. Used in Python, Java 8+ for objects. Stable.